

Pipeline de Observabilidad NOC End-to-End

Manual Técnico de Arquitectura y Manual del Operador

Documentación de Infraestructura — NOC Engineering

Agosto 2026

Pipeline de Observabilidad NOC End-to-End

Manual Técnico de Arquitectura y Manual del Operador

Campo	Valor
Versión del documento	1.0
Stack	Python 3.12 · PostgreSQL 16 + TimescaleDB · Grafana OSS · Docker Compose
Alcance	Ingesta → Almacenamiento en series de tiempo → Visualización → Alertas
Audiencia	Parte 1: Ingenieros de Infraestructura / Parte 2: Operadores NOC Tier 1-2

Tabla de Contenidos

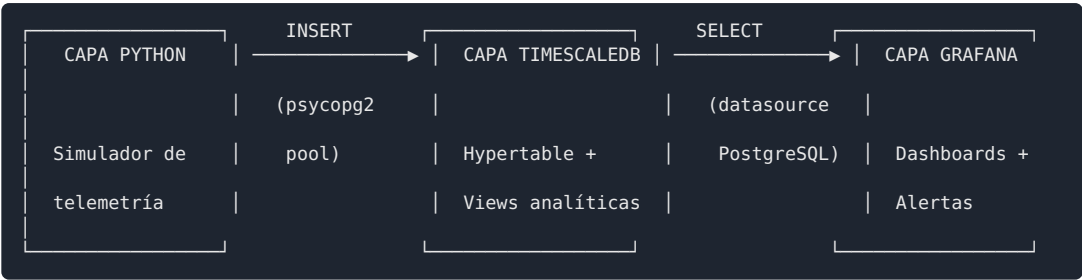
Parte 1 — Arquitectura Técnica y Construcción del Pipeline 1. Visión General de la Arquitectura 2. Capa de Ingesta (Python) 3. Capa de Almacenamiento (TimescaleDB) 4. Capa de Visualización y Despliegue (Grafana + Docker)

Parte 2 — Manual del Operador NOC y Protocolo de Alertas 5. Modo de Uso y Navegación del Dashboard 6. Interpretación de Telemetría 7. Matriz de Alertas y Triage 8. Protocolo de Acción (Playbook)

PARTE 1: Arquitectura Técnica y Construcción del Pipeline

1. Visión General de la Arquitectura

El pipeline implementa un flujo de observabilidad de tres capas desacopladas, comunicadas exclusivamente a través del esquema de datos definido en PostgreSQL/TimescaleDB. Esta separación permite reemplazar cualquier capa sin afectar a las demás — por ejemplo, sustituir el simulador Python por un colector SNMP/Syslog real sin modificar el esquema de base de datos ni los dashboards de Grafana.



1.1 Principio de diseño: sustitución transparente de la fuente

El simulador de telemetría fue diseñado para que su contrato de datos (los campos que escribe en `network_telemetry`) sea **idéntico** al que produciría un colector real de SNMP polling o un receptor de Syslog. Esto significa que la migración de datos simulados a datos de producción es, en teoría, una migración de la Capa 1 únicamente: se reemplaza `simulator.py` por un daemon de polling SNMP (ej. usando `pysnmp` o `easysnmp`) que escribe en el mismo esquema, sin tocar TimescaleDB ni Grafana.

1.2 Resumen de componentes

Componente	Tecnología	Responsabilidad
Generador de telemetría	Python 3.12 (pandas, numpy)	Simula eventos de red con distribución estadística realista
Sink de persistencia	psycpg2 (connection pool)	Inserción de alta frecuencia en PostgreSQL
Motor de series de tiempo	TimescaleDB (extensión de PostgreSQL)	Particionado automático, compresión, retención
Capa de agregación	Vistas SQL	Pre-cálculo de queries recurrentes para Grafana
Visualización	Grafana OSS	Dashboards, paneles, variables de template, alerting
Orquestación	Docker Compose	Despliegue reproducible de TimescaleDB + Grafana

2. Capa de Ingesta (Python)

2.1 Estructura modular

El simulador está dividido en cuatro módulos con responsabilidad única, siguiendo el principio de separación de concerns necesario para que el código sea testeable de forma aislada:

Módulo	Responsabilidad
<code>config.py</code>	Define la topología de red ficticia: 5 dispositivos (<code>NetworkDevice</code> dataclass) con baselines individuales de CPU y latencia
<code>metrics.py</code>	Genera valores numéricos de métricas con distribución estadística y anomalías correlacionadas a la severidad del evento
<code>log_builder.py</code>	Ensambla el registro final: combina métricas + metadata del dispositivo + mensaje tipo syslog

<code>writer.py</code>	Sinks de persistencia intercambiables: CSV, JSON Lines, PostgreSQL
<code>simulator.py</code>	Bucle de ejecución, CLI (<code>argparse</code>), manejo de señales (<code>SIGINT</code>)

Esta modularidad permite, por ejemplo, cambiar la lógica de generación de anomalías en `metrics.py` sin tocar el mecanismo de persistencia en `writer.py`, o añadir un nuevo sink (ej. Kafka producer) sin modificar la lógica de generación de métricas.

2.2 Gaussian Jitter: por qué no usar `random.uniform()`

Un generador ingenuo de métricas de red usando `random.uniform(min, max)` produce una distribución **uniforme**: cualquier valor dentro del rango es igualmente probable. Esto no refleja el comportamiento real de un dispositivo de red, donde las métricas oscilan alrededor de un punto de operación estable (baseline) con variaciones menores, y solo ocasionalmente se desvían significativamente.

El simulador implementa **jitter gaussiano** mediante `numpy.random.normal()`:

```
def _jitter(base: float, pct: float = 0.20) -> float:
    """Aplica jitter gaussiano sobre un valor base (+pct del base)."""
    sigma = base * pct
    return round(max(0.0, np.random.normal(base, sigma)), 2)
```

Justificación técnica:

- La distribución normal concentra la mayoría de los valores generados cerca de la media (`base`), con `sigma` (desviación estándar) controlando la dispersión — en este caso, el 20% del valor base.
- Esto reproduce el comportamiento observado en telemetría real: un router con CPU baseline de 45% oscilará mayormente entre 36-54% bajo carga normal, no saltará aleatoriamente a 90% sin causa.
- El `max(0.0, ...)` previene valores negativos, que no tienen sentido físico para CPU% o latencia.

2.3 Inyección de anomalías correlacionadas a severidad

Para que el dataset simulado sea útil para testear reglas de alerta reales, las métricas no pueden ser independientes de la severidad del evento. `metrics.py` implementa un factor multiplicativo escalonado:

```
def _inject_anomaly(severity: str, base_value: float, ceiling: float) -> float:
    if severity == "WARN":
        factor = random.uniform(1.5, 2.0)
    elif severity == "ERROR":
        factor = random.uniform(2.0, 3.0)
    elif severity == "CRITICAL":
        factor = random.uniform(3.0, 4.5)
    else:
        return _jitter(base_value)
    return round(min(ceiling, base_value * factor), 2)
```

Esto garantiza coherencia semántica: un evento clasificado como `CRITICAL` siempre tendrá métricas numéricas consistentes con esa clasificación (ej. CPU cercano al techo de 100%, o latencia varias veces por encima del baseline del dispositivo). Sin este mecanismo, sería posible generar un evento `CRITICAL` con CPU al 12%, lo cual invalidaría cualquier prueba de las reglas de alerta construidas sobre estos datos.

2.4 Connection Pooling: por qué no abrir una conexión por INSERT

El sink de PostgreSQL (`write_postgres`) utiliza `psycopg2.pool.SimpleConnectionPool` en lugar de abrir y cerrar una conexión TCP por cada evento insertado.

```
def init_pg_pool(dsn: str, minconn: int = 1, maxconn: int = 5) -> None:
    global _connection_pool
    _connection_pool = pg_pool.SimpleConnectionPool(
        minconn=minconn, maxconn=maxconn, dsn=dsn,
    )
```

Justificación técnica:

Enfoque	Costo por INSERT	Comportamiento bajo carga
Conexión nueva por evento	Handshake TCP + autenticación PostgreSQL (~5-15 ms) en cada escritura	Degradación lineal; satura el límite de conexiones del servidor bajo alta frecuencia
Connection Pool	Una vez al arranque; luego solo el costo del <code>INSERT</code> (~1-2 ms)	Throughput estable independiente de la frecuencia de escritura

El pool mantiene un número fijo de conexiones abiertas (`minconn` - `maxconn`) y las reutiliza. Cada llamada a `write_postgres()` toma una conexión libre (`getconn()`), ejecuta el `INSERT` , y la devuelve al pool (`putconn()`) en un bloque `finally` , garantizando que la conexión se libera incluso si el `INSERT` falla.

Nota de escalabilidad: para volúmenes de ingesta significativamente mayores (miles de eventos/segundo, como polling SNMP de cientos de dispositivos), se recomienda migrar de `psycopg2` (síncrono) a `asyncpg` con un pool asíncrono, permitiendo I/O no bloqueante y mayor concurrencia por proceso.

2.5 Manejo de errores y resiliencia del bucle

El bucle principal en `simulator.py` envuelve cada escritura individual en un `try/except` , registrando el error sin abortar la ejecución completa:

```
try:
    writer(record, output_path)
    generated += 1
except Exception as exc:
    log.error(f"Error escribiendo registro: {exc}")
```

Esto es una decisión deliberada de diseño: en un pipeline de observabilidad, la pérdida de un evento individual (ej. por un timeout transitorio de red hacia la base de datos) no debe detener la generación del resto del stream. El error queda registrado en el log de aplicación del propio simulador para diagnóstico posterior.

3. Capa de Almacenamiento (TimescaleDB)

3.1 Por qué TimescaleDB y no PostgreSQL vanilla

TimescaleDB es una extensión de PostgreSQL especializada en datos de series de tiempo. Aporta tres capacidades que PostgreSQL nativo no ofrece de forma automática:

1. **Particionado automático por tiempo** (hypertables) — sin necesidad de gestionar particiones manualmente.

2. **Compresión columnar nativa** — reducciones de espacio del 85-95% en columnas numéricas de telemetría.
3. **Políticas de retención automatizadas** — eliminación de datos antiguos sin jobs externos (`cron` + `DELETE`).

3.2 Diseño del esquema

Tabla de dimensión: `devices`

Almacena metadata estática de cada dispositivo (hostname, IP, rol). Se separa de la tabla de hechos (`network_telemetry`) siguiendo el patrón de modelado dimensional: los atributos que no cambian entre eventos no deben repetirse en cada fila de la serie temporal.

```
CREATE TABLE devices (  
  hostname TEXT PRIMARY KEY,  
  ip INET NOT NULL,  
  role device_role NOT NULL,  
  created_at TIMESTAMPTZ NOT NULL DEFAULT NOW()  
);
```

Tabla de hechos: `network_telemetry`

```
CREATE TABLE network_telemetry (  
  ts TIMESTAMPTZ NOT NULL,  
  hostname TEXT NOT NULL REFERENCES devices(hostname),  
  ip INET NOT NULL,  
  role device_role NOT NULL,  
  severity severity_level NOT NULL,  
  message TEXT,  
  cpu_pct DOUBLE PRECISION CHECK (cpu_pct BETWEEN 0 AND 100),  
  latency_ms DOUBLE PRECISION CHECK (latency_ms >= 0),  
  packet_loss_pct DOUBLE PRECISION CHECK (packet_loss_pct BETWEEN 0 AND 100),  
  interface TEXT,  
  iface_status iface_state,  
  peer_ip INET  
);
```

Decisiones de tipado relevantes:

- `severity`, `role` e `iface_status` son tipos `ENUM` personalizados, no `VARCHAR`. Un `ENUM` ocupa 1-2 bytes en almacenamiento y aplica validación de dominio a nivel de base de datos — un `INSERT` con `severity = 'FATAL'` (valor no definido) falla en la capa de datos, no en la aplicación.
- Los `CHECK` constraints (`cpu_pct BETWEEN 0 AND 100`) previenen inconsistencias de datos incluso si el generador Python tuviera un bug — la base de datos es la última línea de defensa de integridad.

3.3 Conversión a Hypertable

```
SELECT create_hypertable(  
  'network_telemetry',  
  'ts',  
  chunk_time_interval => INTERVAL '1 day',  
  if_not_exists => TRUE  
);
```

`create_hypertable()` transforma la tabla en una estructura particionada automáticamente por la columna `ts`, con un chunk (partición física) nuevo cada 24 horas. Cada chunk es, internamente, una tabla PostgreSQL normal con sus propios índices — las queries que filtran por rango de tiempo

(`WHERE ts BETWEEN ...`) solo tocan los chunks relevantes, evitando escaneos completos de la tabla ante volúmenes de datos que crecen indefinidamente.

3.4 Estrategia de índices

Los índices fueron diseñados en función de los patrones de acceso reales de los paneles de Grafana, no de forma genérica:

```
CREATE INDEX idx_tel_hostname_ts ON network_telemetry (hostname, ts DESC);
CREATE INDEX idx_tel_severity_ts ON network_telemetry (severity, ts DESC);
CREATE INDEX idx_tel_sev_host_ts ON network_telemetry (severity, hostname, ts DESC);
```

Índice	Patrón de query que optimiza
<code>(hostname, ts DESC)</code>	Panel de series temporales filtrado por dispositivo
<code>(severity, ts DESC)</code>	Panel de alertas / conteo de eventos por severidad
<code>(severity, hostname, ts DESC)</code>	Correlación cruzada severidad + dispositivo en dashboards de triage

3.5 Compresión y retención

```
ALTER TABLE network_telemetry SET (
    timescaledb.compress,
    timescaledb.compress_orderby = 'ts DESC',
    timescaledb.compress_segmentby = 'hostname, severity'
);

SELECT add_compression_policy('network_telemetry', compress_after => INTERVAL '7 days');
SELECT add_retention_policy('network_telemetry', drop_after => INTERVAL '90 days');
```

- `compress_segmentby = 'hostname, severity'`: agrupa físicamente los datos comprimidos por estas columnas, optimizando queries que filtran por dispositivo o severidad incluso sobre datos ya comprimidos.
- **Política de compresión** (`compress_after`): los chunks con más de 7 días de antigüedad se comprimen automáticamente en background, sin intervención manual.
- **Política de retención** (`drop_after`): los chunks con más de 90 días se eliminan automáticamente. Este valor debe ajustarse según el SLA de retención de datos del NOC (requisitos de auditoría, compliance, etc.).

3.6 Vistas analíticas para Grafana

En lugar de que cada panel de Grafana contenga lógica SQL compleja y repetida, se centraliza la lógica de agregación en vistas reutilizables:

Vista	Propósito	Consumida por
<code>v_telemetry_ts</code>	Serie temporal completa, tipos casteados a TEXT para compatibilidad con Grafana	Panel de series de tiempo (latencia, CPU)
<code>v_device_latest</code>	Última fila por dispositivo (<code>DISTINCT ON</code>) — snapshot del estado actual	Panel de tabla de estado
<code>v_event_counts</code>	Conteo de eventos agrupado en buckets de 5 minutos	Paneles de tendencia de volumen de eventos
<code>v_recent_anomalies</code>	Eventos WARN+ de la última hora	Tablas de alerta / triage

```
CREATE OR REPLACE VIEW v_device_latest AS
SELECT DISTINCT ON (hostname)
    ts, hostname, severity::TEXT AS severity,
    cpu_pct, latency_ms, packet_loss_pct, iface_status::TEXT AS iface_status
FROM network_telemetry
ORDER BY hostname, ts DESC;
```

`DISTINCT ON (hostname)` combinado con `ORDER BY hostname, ts DESC` es el patrón idiomático de PostgreSQL para obtener “la fila más reciente por grupo” — más eficiente que una subquery con `ROW_NUMBER() OVER (PARTITION BY ...)` para este caso de uso específico.

4. Capa de Visualización y Despliegue (Grafana + Docker)

4.1 Infrastructure as Code: justificación

Todo el stack (TimescaleDB + Grafana, incluyendo datasources y dashboards) se define declarativamente en archivos versionables (`docker-compose.yml` + YAML/JSON de provisioning). Esto elimina la configuración manual vía UI, que no es reproducible ni auditable.

Beneficios operativos concretos:

- Un nuevo ambiente (staging, DR) se levanta con `docker compose up -d` sin pasos manuales.
- Los cambios de configuración quedan en control de versiones (Git), con historial de auditoría.
- No existe “drift” de configuración entre ambientes — todos parten del mismo YAML.

4.2 Orquestación con Docker Compose

```
services:
  timescaledb:
    image: timescale/timescaledb:latest-pg16
    environment:
      POSTGRES_DB: noc
      POSTGRES_USER: noc_user
      POSTGRES_PASSWORD: secret
    networks: [noc_net]

  grafana:
    image: grafana/grafana-oss:latest
    depends_on: [timescaledb]
    volumes:
      - grafana_data:/var/lib/grafana
      - ./grafana/provisioning:/etc/grafana/provisioning
    networks: [noc_net]
```

Puntos críticos de la configuración:

- Ambos servicios comparten la red `noc_net` (bridge network de Docker). Grafana resuelve TimescaleDB por nombre de servicio (`timescaledb:5432`), no por `localhost` — un error común en despliegues Dockerizados.
- `depends_on` garantiza el orden de arranque, aunque no espera a que PostgreSQL esté completamente listo para aceptar conexiones (para eso, en producción, se recomienda un healthcheck explícito).
- El volumen `./grafana/provisioning` monta la configuración declarativa dentro del contenedor, donde Grafana la lee automáticamente al iniciar.

4.3 Provisioning automático de Datasource

```
datasources:
  - name: TimescaleDB-NOC
    type: postgres
    uid: timescaledb_noc
    url: timescaledb:5432
    database: noc
    user: noc_user
    secureJsonData:
      password: secret
    jsonData:
      sslmode: disable
      postgresVersion: 1600
      timescaledb: true
    isDefault: true
```

Dos detalles no obvios pero críticos:

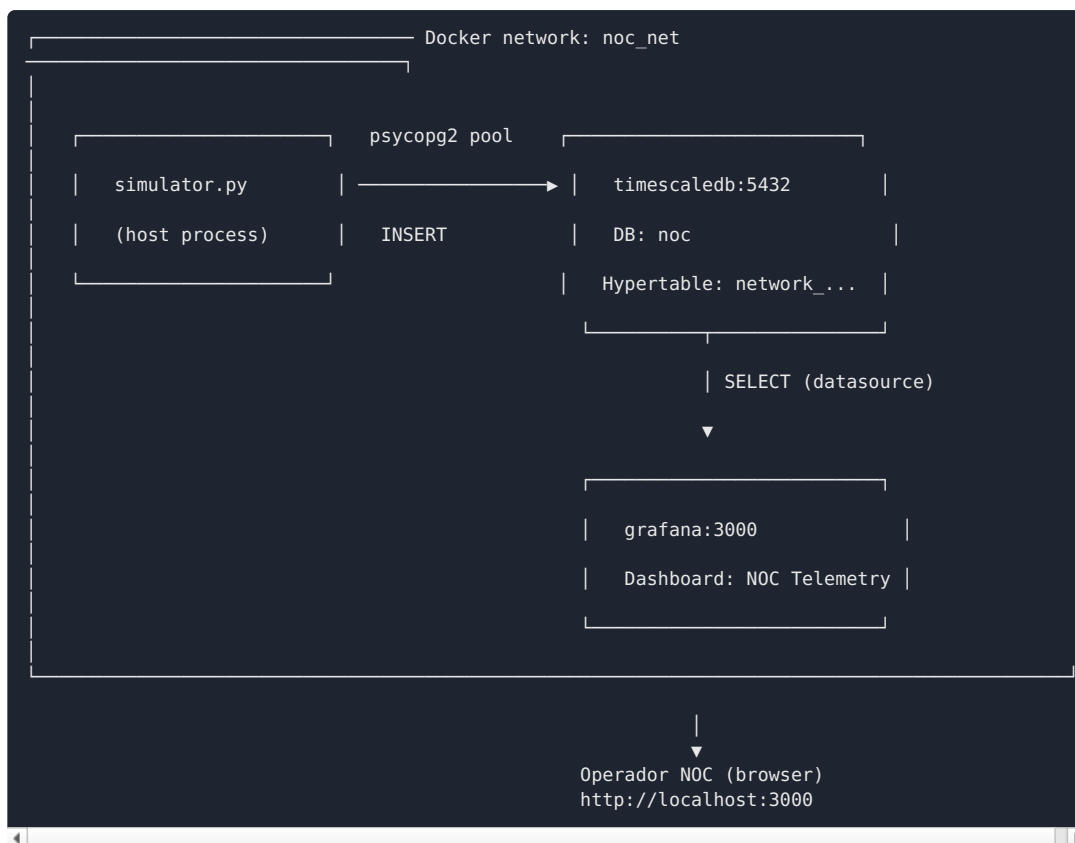
- `uid: timescaledb_noc` es un identificador **fijo**, no autogenerado. El JSON del dashboard referencia este UID directamente en cada panel. Sin un UID fijo, cada reimportación del dashboard generaría un UID aleatorio nuevo, rompiendo la vinculación panel↔datasource (“datasource not found”).
- `timescaledb: true` en `jsonData` activa el motor de queries optimizado de Grafana para hypertables, habilitando macros como `$__timeFilter()` y `time_bucket()` de forma nativa en el editor de queries.

4.4 Provisioning automático de Dashboard

```
providers:
  - name: NOC Dashboards
    type: file
    updateIntervalSeconds: 30
    options:
      path: /etc/grafana/provisioning/dashboards
```

El archivo `noc_telemetry.json` (dashboard completo, exportado desde Grafana o escrito directamente) se coloca en esa ruta. Grafana lo detecta y carga automáticamente al iniciar, y vuelve a leerlo cada 30 segundos si el archivo cambia en disco — permitiendo iterar sobre el dashboard editando el JSON directamente, sin pasar por la UI.

4.5 Diagrama de despliegue final



PARTE 2: Manual del Operador NOC y Protocolo de Alertas

5. Modo de Uso y Navegación del Dashboard

5.1 Acceso

El dashboard principal se encuentra en Grafana bajo el nombre **“NOC — Network Telemetry”**, accesible en <http://<host-grafana>:3000>. El refresco automático está configurado a **10 segundos** por defecto, ajustable desde el selector de tiempo en la esquina superior derecha (opciones: 5s / 10s / 30s / 1m / 5m).

5.2 Estructura del dashboard

El dashboard está organizado en tres secciones apiladas verticalmente, en orden de prioridad operativa:

Orden	Sección	Paneles	Propósito operativo
1	⚡ Critical Events — Last 15 min	Stat de conteo + Tabla de eventos	Lo primero que el operador debe mirar al iniciar turno
2	📊 Time Series — Latency & CPU	Gráfico de series de tiempo por dispositivo	Detección de tendencias y patrones de degradación
3	🖼️ Device Snapshot	Tabla de estado actual	Foto instantánea del

5.3 Panel de estado actual de enlaces (Device Snapshot)

Muestra la última lectura registrada de cada uno de los dispositivos monitoreados, ordenada automáticamente con los estados más críticos en la parte superior (**CRITICAL** → **ERROR** → **WARN** → **INFO**).

Codificación de color (aplicada automáticamente vía thresholds):

Columna	Verde	Amarillo	Rojo
CPU %	< 70%	70-90%	> 90%
Latencia (ms)	< 50 ms	50-200 ms	> 200 ms
Estado de interfaz	UP	—	DOWN

5.4 Panel de tendencias temporales (Time Series)

Muestra la evolución de **latencia** (eje izquierdo, ms) y **CPU** (eje derecho, %) por dispositivo, en el rango de tiempo seleccionado. El operador puede:

- Usar el dropdown **\$hostname** en la parte superior del dashboard para filtrar por uno o varios dispositivos específicos (por defecto: todos).
- Hacer zoom arrastrando sobre una región del gráfico para inspeccionar un intervalo específico con mayor resolución temporal.
- Pasar el cursor sobre cualquier punto para ver el valor exacto de todos los dispositivos en ese instante (tooltip modo “multi”).

5.5 Panel de carga de CPU

La carga de CPU se visualiza tanto en el panel de series de tiempo (tendencia histórica) como en el snapshot de estado (valor actual). Un patrón a vigilar: los **routers core** (**core-rtr-01** , **core-rtr-02**) tienen un baseline de CPU más alto (~40-45%) que los **switches de acceso** (~15%), por lo que el mismo valor absoluto de CPU% representa un nivel de estrés muy distinto según el rol del dispositivo — ver sección 6.3.

6. Interpretación de Telemetría (Simulación SNMP)

Nota de equivalencia: los datos consumidos en este dashboard son estructuralmente equivalentes a los que produciría un colector SNMP real (polling de OIDs como **ifOperStatus** , **cpmCPUTotal5minRev**) o un receptor de Syslog (traps de **linkDown** / **linkUp**). El operador debe interpretarlos con el mismo criterio que aplicaría a telemetría de producción.

6.1 Incremento súbito vs. incremento gradual en latencia

Patrón	Lectura típica	Causa probable	Urgencia
Súbito (salto de >3x el baseline en un solo intervalo de polling)	5 ms → 180 ms en < 1 minuto	Falla de enlace físico, saturación instantánea de buffer, ataque DDoS, cambio de ruta BGP no óptimo	Alta — investigar de inmediato

Gradual (incremento sostenido a lo largo de varios buckets de tiempo)	5 ms → 15 ms → 30 ms → 45 ms a lo largo de 30 min	Saturación progresiva de ancho de banda, degradación de hardware, incremento orgánico de tráfico	Media — monitorear tendencia, escalar si continúa la pendiente
--	---	--	--

Regla práctica: un salto súbito indica un evento discreto (algo se rompió); una pendiente gradual indica un proceso continuo (algo se está agotando). El primero requiere respuesta inmediata; el segundo requiere planificación de capacidad.

6.2 Correlación Packet Loss ↔ Saturación de CPU

La pérdida de paquetes (`packet_loss_pct`) rara vez es un fenómeno aislado. Su correlación con CPU depende del **rol** del dispositivo:

Rol de dispositivo	Mecanismo de correlación	Interpretación operativa
Router core	CPU alto → el motor de forwarding no alcanza a procesar la tabla de rutas a línea de velocidad → paquetes descartados en cola	Packet loss + CPU alto simultáneo en un core router = degradación de plano de control , riesgo de afectar múltiples segmentos downstream
Switch de acceso	CPU alto generalmente NO correlaciona con packet loss (el forwarding L2 es mayormente en hardware/ASIC)	Packet loss en un access switch con CPU normal sugiere problema físico de capa 1 (cable, transceiver, duplex mismatch) más que saturación de procesamiento

Regla de triage: si observas packet loss + CPU alto en un `core-rtr-*`, trátalo como incidente de plano de control con potencial impacto amplio. Si observas packet loss con CPU normal en un `access-sw-*`, sospecha primero de capa física antes de escalar como problema de capacidad.

6.3 Interpretación de CPU según rol del dispositivo

No existe un umbral único de CPU válido para todos los dispositivos — depende del baseline operativo normal de cada rol:

Rol	Baseline normal	CPU “alto” en contexto
<code>core-router</code>	~40-45%	> 85% es preocupante (poco margen para picos de convergencia de rutas)
<code>distribution-sw</code>	~22-25%	> 80% indica carga anómala (normalmente muy por debajo)
<code>access-sw</code>	~15%	> 70% ya es una desviación significativa del comportamiento esperado

7. Matriz de Alertas y Triage

Esta matriz define los umbrales técnicos que clasifican un evento en uno de cuatro niveles de severidad operativa. Estos umbrales deben usarse como base para configurar las reglas de alerta en Grafana (Fase 4 del pipeline).

Nivel	CPU %	Latencia	Packet Loss	Estado de interfaz	SLA de respuesta
-------	-------	----------	-------------	--------------------	------------------

WARNING	70-85%	50-100 ms	0.5-1%	Flapping (UP/DOWN intermitente)	Monitorear — sin acción inmediata obligatoria
MINOR	85-90%	100-150 ms	1-3%	—	Revisar dentro de 30 min
MAJOR	90-95%	150-200 ms	3-10%	DOWN en enlace redundante (con backup activo)	Atender dentro de 15 min
CRITICAL	> 95%	> 200 ms	> 10%	DOWN en enlace sin redundancia / equipo core inalcanzable	Respuesta inmediata (< 5 min)

Notas de aplicación de la matriz:

- Un solo indicador en nivel CRITICAL es suficiente para clasificar el evento completo como CRITICAL, independientemente del estado de los demás indicadores (regla de “el peor indicador domina”).
- Para `core-router`, considerar aplicar los umbrales de CPU un nivel más estricto (ej. tratar 80-85% como MAJOR en lugar de WARNING) dado su menor margen operativo — ver sección 6.3.
- Los umbrales de esta tabla corresponden directamente a los `thresholds` configurados en los paneles de Grafana (`fieldConfig.defaults.thresholds`) y deben mantenerse sincronizados si se ajustan en el dashboard.

8. Protocolo de Acción (Playbook)

8.1 Flujo de decisión al detectar un evento CRITICAL



8.2 Pasos detallados

Paso 1 — Validación de falso positivo

Antes de escalar, confirmar que el evento no es un pico transitorio de un único ciclo de polling.

- Revisar el panel de series de tiempo: ¿el valor volvió a rango normal en el siguiente punto de datos, o se mantiene elevado?
- Si el evento desaparece en la siguiente lectura (10-20 segundos después con el refresh configurado) y no se repite, clasificar como ruido transitorio y registrar en bitácora sin escalar.
- Si persiste durante 2 o más ciclos de refresco consecutivos, proceder al Paso 2.

Paso 2 — Aislamiento del alcance

Usar el filtro `$hostname` del dashboard para determinar si el evento afecta a:

- **Un solo dispositivo** → probable falla localizada (hardware, enlace específico).
- **Múltiples dispositivos del mismo segmento** → posible falla de upstream compartido (switch de distribución, enlace troncal).
- **Dispositivos core-*** → tratar con máxima prioridad independientemente del nivel de la matriz — el radio de impacto de un core router es sistémicamente mayor.

Paso 3 — Clasificación

Aplicar la Matriz de Alertas y Triage (Sección 7) al conjunto de métricas observadas (CPU, latencia, packet loss, estado de interfaz) para asignar el nivel: WARNING / MINOR / MAJOR / CRITICAL.

Paso 4 — Acción según nivel

Nivel	Acción del operador
WARNING	Anotar en bitácora de turno. No requiere notificación.
MINOR	Anotar en bitácora + revisar nuevamente en 30 min. Si escala a MAJOR, pasar a esa fila.
MAJOR	Notificar a Tier 2 vía canal de guardia (Slack/Teams del NOC). Mantener el panel de tabla de eventos abierto para seguimiento en tiempo real.
CRITICAL	Escalar inmediatamente según cadena de escalamiento del NOC (ver 8.3). Iniciar bitácora de incidente formal con timestamp de detección.

Paso 5 — Documentación y seguimiento

Todo evento MINOR o superior debe registrarse con: timestamp de detección (según columna `ts` del panel de tabla), dispositivo(s) afectado(s), métricas observadas en el momento de detección, nivel asignado, y acción tomada. Esta bitácora es la fuente de verdad para el post-mortem y para el cálculo de métricas de SLA del NOC (MTTA, MTTR).

8.3 Cadena de escalamiento (CRITICAL)

- 1. **Tier 1 (operador de turno):** detecta, valida falso positivo, aísla alcance, clasifica.
- 2. **Tier 2 (ingeniero de guardia):** notificado automáticamente en eventos CRITICAL. Asume investigación técnica profunda.
- 3. **Tier 3 / Arquitecto de infraestructura:** escalado si el Tier 2 no logra mitigar dentro de la ventana de SLA definida, o si el incidente involucra múltiples sistemas core simultáneamente.

El operador Tier 1 **nunca** debe intentar remediación directa sobre equipos core sin autorización de Tier 2 — su función en el protocolo es detección, clasificación y escalamiento, no remediación de infraestructura crítica.

Anexo: Referencia Rápida de Comandos Operativos

Necesidad	Dónde en el dashboard
Ver eventos críticos activos ahora	Panel superior “CRITICAL / DOWN Events” (Stat)
Ver historial de eventos críticos últimos 15 min	Tabla “Critical & DOWN Event Log”
Filtrar por un dispositivo específico	Dropdown <code>\$hostname</code> (esquina superior del dashboard)
Ver tendencia de latencia/CPU de las últimas horas	Ajustar time range picker (esquina superior derecha) a “Last 3 hours” o superior
Ver estado actual de todos los equipos	Tabla inferior “Device Current State”
Cambiar frecuencia de auto-refresh	Selector junto al time range picker (5s/10s/30s/1m/5m)

Fin del documento — Pipeline de Observabilidad NOC End-to-End v1.0